

Aria feature guide

Buyer-side B2B parts negotiation bot. Aria is an AI procurement assistant. She discloses that once at the start of a desk. The numbers are not AI — the deal engine owns price, the ■10,00,000 ceiling, and walk-away.

Product	Aria · SKODA sourcing desk
UI	http://localhost:3000
Ops	http://localhost:3000/ops
API	127.0.0.1:8001 when port 8000 is taken
Audience	SKODA buyers and ops. Never vendor-facing.
Date	10 September 2026

This PDF is the feature record for the product and the four ops dashboards. It is not a markdown file. Secrets, API keys, and walk-away prices are omitted on purpose.

1. What Aria is

Aria sits on the SKODA side of a parts RFQ. One session is one part–vendor line. She greets, qualifies the vendor contact, checks whether the quoted order total is even eligible for automated negotiation, opens with a deterministic anchor, then bargains inside a hard ceiling. If the desk cannot close inside policy she hands off to a human buyer.

Non-negotiable product rules

- Aria is an AI and must say so once, at the start of the desk.
- The deal engine (`backend/app/deal_engine.py`) has zero LLM imports.
- The ■10,00,000 gate is on order total ($\text{unit price} \times \text{quantity}$), constant `CEILING = 1,000,000`.
- Walk-away / max acceptable is never spoken, trained, or shown to the vendor.
- Insights, reasoning, labels, and ops dashboards are SKODA-only.

2. Home: Open RFQs

Route `/`. Lists live catalog RFQs with part, vendor, unit quote, and quoted total. If quoted total is above ■10L the row says so before anyone opens a desk.

Negotiate	Opens <code>/chat?part={id}</code> with a live vendor. Human types on the vendor side.
Test	Opens <code>/chat?part={id}&demo=1</code> . A vendor bot replies so buyers can watch Aria without a live counterpart.
Ops	Opens the overall status dashboard at <code>/ops</code> .

Test personas (demo=1)

- headlight-lh — cooperative vendor.
- chassis-frame-front — over-ceiling quote; eligibility should fail automated close.
- rear-subframe — ceiling-boundary desk.
- wiring-harness-cabin — vague vendor; Aria must qualify before pricing.

In Test mode the composer is disabled. The UI loops vendor-bot → Aria automatically until the persona is done.

3. Negotiation desk

Route `/chat`. Three columns: part dossier, chat + deal rail, SKODA insights. Header shows live vs mock model, stage, and links back to RFQs and Ops.

Desk surfaces

- Part dossier — part name, vendor, quantity, quoted unit, spec blurb.
- Deal rail — vendor ask, Aria bid, stage. Numbers come from the engine, not free text.
- Chat — streamed Aria replies. Vendor-facing text never includes reasoning or walk-away.
- Insights panel — situation, tactic, sentiment, internal reasoning. SKODA only.
- Handoff — shown when the engine or policy requires a human buyer.

Session stages

greet_and_disclose → qualify → price_eligibility_check → open_offer → negotiate → agreement / handoff / closed.

4. Deal engine

Deterministic. The LLM may propose a unit price; the engine decides whether that number may be said. Aria represents the buyer: she opens low and must never agree to pay above max acceptable unit price.

Anchor	Opens ~8% below target, never above the vendor quote.
Concessions	Diminishing schedule of the room between target and ceiling.
■10L gate	If unit × qty > 1,000,000 the line is not auto-closed; handoff.
Hostile language	Vendor abuse / fraud language can force handoff.
Hold tactics	Named tactics the engine will not undercut on the same round.

5. Dual models

Commercial complete() / propose_price uses a reasoning model (default o4-mini, effort medium). Streamed vendor-facing text uses a chat model (default gpt-4o). o4-mini cannot be fine-tuned. Fine-tune target is gpt-4o-mini for voice only. If the reasoning model is unavailable that turn falls back to the chat model. GPT-5.4+ Chat Completions with tools forces reasoning_effort=none.

After a successful job, the promoted chat model is stored in backend/data/finetune/active_model.json and resolved_chat_model() serves it. Do not plant fake ft: models in that file.

6. Strategy playbook and lesson memory

Each turn is classified into a situation. Coaching and a tactic that has not already failed are injected into the prompt. Aria does not train weights from this. Walk-away is never stored in a lesson.

- ceiling_probe — fishing for a max. Hold. Do not name a ceiling.
- firm_floor — vendor named a cost. One visible move or a terms trade.
- stalling — no price cut while they check with a manager.
- leadtime_gate / quality_gate — trade calendar or warranty, hold unit price.
- package_ask — answer terms, MOQ, lead time together.
- stuck_repeat — last tactic failed; change the lever.
- in_range_push — payable number, one push, then close.
- cooperative_move / ready_to_close / opening / price_fight.

Closed desks write a short lesson (JSON file + SQLite). Later similar desks retrieve up to three prior tactics. API: GET /api/lessons.

7. Audit database

Always on, even with USE_IN_MEMORY=true. Local SQLite at backend/data/aria_audit.db (gitignored). Postgres dual-write happens when a pool exists. Nothing in this database is sent to the vendor.

turns	Every vendor and Aria message plus tactic, situation, reasoning, model.
insights	Per-round SKODA insight snapshot.

lessons	Reusable close notes without walk-away.
session_snapshots	Latest public-ish session payload for live lists.
labels	train / reject quality for fine-tune export.
ft_jobs	OpenAI fine-tune job id, status, promoted model.

8. Fine-tune pipeline

Live, auto on by default. After a desk closes, and every 90 seconds in the FastAPI lifespan, the pipeline ticks. It labels closed desks, exports JSONL when asked, and at 20 train voice examples can start a supervised job on gpt-4o-mini. Pytest cannot spend money: ARIA_ALLOW MOCK=1 blocks a live client unless force=true and a client is injected.

Label rules (train vs reject)

- Reject if the desk is not closed / agreement.
- Reject if the transcript is too short to be a voice example.
- Reject if Aria leaked walk-away, named a ceiling, named a target, or used banned assistant phrases.
- Train only when the voice turns are clean. Brain/reasoning JSONL is exported separately and is not the OpenAI job.

Controls and config

OPENAI_FINETUNE_AUTO	true — tick may submit when ready.
OPENAI_FINETUNE_BASE	gpt-4o-mini (voice). Not o4-mini.
OPENAI_FINETUNE_MIN_EXAMPLES	20 train voice examples.
Dedup	Dataset fingerprint. Will not resubmit the same set.
CLI	python -m app.finetune preview export tick submit status

HTTP for the pipeline

GET /api/finetune/preview	Counts only. Does not start a job.
GET /api/finetune/status	Labels, jobs, active chat model, readiness.
POST /api/finetune/tick	Label, poll open job, submit if auto+ready.
POST /api/finetune/export	Write JSONL under backend/data/finetune. No job.
POST /api/finetune/submit	Force tick including a paid job if ready. Dashboard confirms first.

9. Overview dashboard

Route /ops. Overall status, health checks, and a snapshot of every other desk. Polls GET /api/ops/overview every four seconds.

Status cards

- API — up/down, company, bind port.
- LLM — live provider vs mock. Mock means fixture text, not production voice.
- Reasoning — o4-mini (or configured). Never fine-tuned.
- Fine-tune — ready, or train_examples / min_examples. Auto on/off.
- Sessions, turns, lessons, labels — SQLite counts.

Health checks panel

- FastAPI /health responds.
- Live OpenAI provider (not mock).
- Chat model set.
- Reasoning model set.
- Fine-tune auto tick enabled.
- No open fine-tune job (watch if a job is in flight).
- Catalog parts loaded.
- Audit SQLite path shown at the foot of the panel.

Controls

- Recheck — immediate refetch of overview.
- Recent desks list — open the selected session on the Data dashboard.
- Nav — Overview, Data, Logs, Fine-tune, Open RFQs.

10. Data dashboard

Route `/ops/data`. Live visibility of collected desks, lessons, and labels. Polls sessions, lessons, and labels every four seconds. Query `?session=` opens a desk from Overview.

What is visible

- Counts — desks, lessons, train labels, reject labels.
- Filter — part, vendor, stage, session id.
- Session list — stage, rounds, Aria bid, vendor ask, contact name.
- Desk detail — turns, tactics, situations, SKODA reasoning, train/reject reasons.
- Playbook lessons — part, situation, outcome, tactics, note. No walk-away.

Controls

- Refresh — pull the live feed now.
- Click a session — load GET `/api/sessions/{id}/log`.
- Filter field — client-side subset of the live list.

11. Logs dashboard

Route `/ops/logs`. Cross-desk audit stream from GET `/api/logs/recent` (last 80 turns). Polls every four seconds.

- Cards — recent turns, assistant count, vendor count.
- Role filter — all / assistant / vendor.
- Search — content, reasoning, tactic, situation, model, session id.
- Each row — role, short session id, stage, model, tactic, situation, message, reasoning, timestamp.
- Refresh — immediate pull.

12. Fine-tune dashboard

Route `/ops/finetune`. Status, controls, testing, labels, and jobs. Polls `GET /api/finetune/status` and `GET /health` every five seconds.

Status cards

- Ready — yes, or not yet, with `train_examples` / `min_examples`.
- Auto — on/off. Warns if the LLM provider is mock.
- Chat model — current voice model, including a promoted `ft: id` when present.
- Open job — none, or OpenAI status + job id.
- Train labels, reject labels, valid split, skipped desks.

Controls

Tick pipeline	POST <code>/api/finetune/tick</code> . Labels new desks, polls an open job, submits only if auto and ready.
Export JSONL	POST <code>/api/finetune/export</code> . Writes <code>voice.jsonl</code> , <code>voice_valid.jsonl</code> , <code>brain.jsonl</code> , <code>manifest.json</code> . No paid job.
Force submit	POST <code>/api/finetune/submit</code> . Disabled until ready. Confirm dialog: this spends money.
Refresh status	GET status + health now.
Run checks	Testing: GET <code>/health</code> , GET <code>/api/finetune/status</code> , GET <code>/api/finetune/preview</code> . Does not tick or submit.

Testing panel

Run checks reports pass/fail for Health (API up and not mock), Status (auto flag, example counts, active chat model), and Preview (train/valid/skipped). It will not start an OpenAI job. Use Tick to label. Use Force submit only when the desk is deliberately ready to spend.

Tables

- Labels — session prefix, quality, outcome, voice example count, reject reasons.
- Jobs — status, job id, base model, promoted fine-tuned model, error if any.

13. Ops navigation and visual system

All four dashboards share the sourcing-desk look. Do not invent a second visual language.

Colors	pit #14181F, bone #EEEEAE2, amber #E09A3D, stitch #8A6A4B, ink #1A1714, ceiling #C0392B, deal #2E6B4F, fog #C9CDD3
Type	Syne display, Figtree sans, IBM Plex Mono
Corners	Sharp. Dossier, not consumer chat.
Entry	Home Ops button, chat header Ops, /ops nav.

14. HTTP map (ops + product)

GET /health	Provider, mock flag, chat and reasoning models, company, port.
GET /api/catalog	RFQs and public part listings.
POST /api/sessions	Start a desk for a part_id.
POST /api/chat/stream	SSE tokens, session, insights, price, done.
POST /api/demo/vendor-turn	Vendor bot for Test mode.
GET /api/sessions/{id}	Public session + transcript.
GET /api/sessions/{id}/insights	SKODA insight log for a live session.
GET /api/sessions/{id}/log	Audit turns, insights, lessons, label.
GET /api/lessons	Playbook memory.
GET /api/labels	All train/reject rows. Optional ?quality=.
GET /api/logs/sessions	Session snapshots for the Data dashboard.
GET /api/logs/recent	Last 80 turns for the Logs dashboard.
GET /api/ops/overview	Health + counts + recent desks + pipeline snapshot.

Frontend proxies /backend/:path* to the API using BACKEND_PORT or BACKEND_INTERNAL_URL. Client fetch base is NEXT_PUBLIC_API_URL or /backend.

15. How to run

Backend (port 8000 is often taken by an unrelated service — do not kill it)

```
cd backend
$env:USE_IN_MEMORY = "true"
$env:BACKEND_PORT = "8001"
$env:BACKEND_HOST = "127.0.0.1"
python -m app.main
```

Frontend

```
cd frontend
$env:BACKEND_PORT = "8001"
$env:BACKEND_INTERNAL_URL = "http://127.0.0.1:8001"
npm run dev
```

UI http://localhost:3000 · API http://127.0.0.1:8001/health · tests: cd backend; python -m pytest -q

16. What this product does not do

- It does not fine-tune o4-mini.
- It does not put walk-away, target, or authorization limits in vendor chat or training voice.
- It does not auto-submit a paid job from Run checks or Export.
- It does not replace a human buyer when the █10L total gate trips or the engine hands off.
- It does not document or print OPENAI_API_KEY.

End of feature guide. Dashboards: /ops · /ops/data · /ops/logs · /ops/finetune.